

POLITECNICO
MILANO 1863

Best Bike Paths, Luca Suzani, Alberto Spezzapria, Alessio Pesavento

ITD Document

Deliverable:	ITD
Title:	ITD Document
Authors:	Luca Suzani, Alberto Spezzapria, Alessio Pesavento
Version:	1.0
Date:	01-February-2026
Download page:	https://github.com/Alessio01103/PesaventoSpezzapriaSuzani.git
Copyright:	Copyright © 2026, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Contents

Table of Contents 3
List of Figures 4
List of Tables 4

1 Introduction **6**

2 Implemented Requirements and Functions **7**

3 Adopted Development Frameworks and Technologies **8**
3.1 Adopted Programming Languages 8
3.2 Middleware 9

4 Source Code Structure **10**

5 Testing **11**
5.1 Testing 11
5.1.1 Geospatial Path Recognition 11
5.1.2 Community Integrity and Data Protection 11
5.1.3 Privacy and Workflow Consistency 12

6 Installation Instructions **13**

7 Effort Spent **14**

List of Figures

List of Tables

5.1 Geospatial Similarity Test Cases 11
5.2 Community Integrity Test Cases 12
5.3 Privacy and Workflow Logic Test Cases 12

7.1 Hours of work spent by each author on app implementation and document preparation . 14

Chapter 1

Introduction

Chapter 2

Implemented Requirements and Functions

Chapter 3

Adopted Development Frameworks and Technologies

3.1 Adopted Programming Languages

For the development of the **BestBikePaths** platform, the architecture of choice is a **Progressive Web App (PWA)**. This technology bridges the gap between traditional web applications and native mobile software. By utilizing a web app manifest, the platform can be installed directly onto a user's device, providing a native-like cross-platform experience from a single codebase. This approach significantly optimizes the development lifecycle, testing procedures, and long-term maintenance.

While PWAs can occasionally encounter limitations regarding offline data persistence and deep hardware integration, current technological standards provide robust solutions to these challenges:

- **Connection Resilience:** Through the implementation of **Service Workers**, the application can intercept network requests and manage advanced caching strategies, ensuring that core functionalities remain available even in offline environments.
- **Hardware Integration:** Regarding access to system sensors (such as the accelerometer, camera, or microphone), frameworks like **Capacitor** offer a solution by providing a standardized interface between web code and the native APIs of iOS and Android, effectively addressing interoperability constraints.

TypeScript: Advantages and Disadvantages

The application was developed using **TypeScript**, a superset of JavaScript that introduces static typing. Its adoption within the project is justified by the following technical analysis:

Advantages

- **Static Typing and Reliability:** TypeScript allows for the detection of type-related errors during the compilation phase rather than at runtime. This leads to more stable code and a significant reduction in production bugs.
- **Maintainability and Scalability:** The use of explicit interfaces and custom types ensures the codebase is self-documenting. This facilitates collaborative development and simplifies the process of scaling the application as new features are added.
- **Enhanced Developer Tooling:** TypeScript provides superior IDE support, including advanced autocompletion, precise navigation, and safer refactoring, which collectively increase development velocity.

Disadvantages

- **Transpilation Requirement:** TypeScript cannot be executed natively by web browsers. It requires a compilation (transpilation) step into JavaScript, which adds complexity to the automated build pipeline.
- **Learning Curve:** Proper implementation of strict typing and advanced architectural patterns requires a higher level of technical expertise compared to standard JavaScript.
- **Initial Verbosity:** The requirement to explicitly define data structures results in more boilerplate code during the early stages of development, which can slightly increase the time required for rapid prototyping.

3.2 Middleware

Serverless Logic and Infrastructure: Supabase Edge Functions

To handle server-side business logic and third-party integrations, the project utilizes **Supabase Edge Functions**. These are serverless TypeScript functions executed across a distributed Edge Network—a global infrastructure of nodes (Points of Presence) designed to process data at the "edge" of the internet, physically closer to the end-user than traditional centralized cloud servers.

The adoption of this architecture provides several strategic advantages and some trade-offs:

Advantages

- **Minimal Latency:** By executing logic on the edge node nearest to the user, the application significantly reduces data travel time, ensuring a highly responsive experience for mobile users in motion.
- **Enhanced Security:** Edge Functions provide a secure environment to manage private API keys and sensitive validation logic, keeping them isolated from the client-side environment.
- **Automated Scalability:** The serverless nature of the infrastructure allows it to scale dynamically with traffic, removing the need for manual server provisioning or maintenance.
- **Native Ecosystem Integration:** Full compatibility with Supabase Auth and PostgreSQL enables the seamless implementation of complex workflows, such as secure data processing and automated notifications.

Disadvantages

- **Cold Starts:** Infrequently used functions may encounter a minor delay during the initial invocation as the execution environment warms up.
- **Resource Limits:** Compared to traditional dedicated servers, Edge Functions operate within stricter memory and execution time constraints.
- **Distributed Debugging:** Testing and monitoring logic across a global network can be more complex than managing a centralized, monolithic backend.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 4

Source Code Structure

The codebase is engineered to promote modularity and scalability, ensuring that individual components can be updated or reused without compromising the integrity of the entire system. A strict separation of concerns has been enforced by organizing the project into distinct directories, effectively decoupling UI elements, business logic, and external service integrations. All core PWA source code is contained within the `src` directory, structured as follows:

Components: This directory contains both bespoke project components and those derived from the **shadcn/ui** library. The `ui` subfolder is reserved for the foundational shadcn library components, while the `app` subfolder and the root of the directory house custom-built elements.

Hooks: This folder contains custom React Hooks, which encapsulate reusable logic to maintain clean components. They are specifically utilized for **Device Detection** (adapting the UI for mobile vs. desktop) and **Toast Notifications** (managing system feedback).

Pages: This directory follows a file-based routing logic, containing the main views of the application. The `app` subfolder organizes core views such as the **Dashboard**, **ExplorePaths**, and **RecordTrip**, ensuring a clear mapping between URL paths and UI controllers.

Stores: This directory houses global state management logic (e.g., via Zustand). Key files include `useAuthStore.ts` for authentication states and `useDebugStore.ts` for development-time performance monitoring.

Lib: Dedicated to utility functions and service configurations. It includes `utils.ts` for shared helpers and specialized modules like `weatherService.ts`, which abstracts the logic for processing external meteorological data.

Chapter 5

Testing

5.1 Testing

The implementation has undergone a rigorous testing phase focusing on the geospatial algorithms, data integrity constraints, user permission logic (Row Level Security), and privacy-aware workflows. The following subsections describe the main validation scenarios designed to ensure correctness, usability, and protection of community data.

5.1.1 Geospatial Path Recognition

This module verifies the system's ability to identify whether a newly recorded activity matches an existing path in the database using the PostGIS-powered `check_path_similarity` RPC function.

ID	Test Scenario	Expected Outcome	Result
GEO-01	**Perfect Match:** User records an activity that strictly follows the geometry of an existing public path within the 50m tolerance buffer.	The system identifies the existing path ID. The UI displays a "Match Found" dialog prompting the user to link the activity to the existing path instead of creating a new one.	**Pass**
GEO-02	**Partial Match / Deviation:** User records a track that overlaps with an existing path for 40% of the length but deviates significantly for the rest.	The system returns no match (null). The UI prompts the user to create a new path with a new name and description.	**Pass**
GEO-03	**Reverse Direction:** User traverses an existing path starting from the end point to the start point.	The algorithm recognizes the path regardless of the traversal direction.	**Pass**

Table 5.1: Geospatial Similarity Test Cases

5.1.2 Community Integrity and Data Protection

These tests validate the logic governed by the `get_path_community_usage` function and RLS policies, ensuring that public resources used by the community cannot be deleted or modified in ways that would compromise data integrity.

ID	Test Scenario	Expected Outcome	Result
SEC-01	**Safe Deletion:** An owner attempts to delete a path with 0 associated activities from other users.	The deletion proceeds successfully. The path and the owner's personal activities are removed (cascade delete).	**Pass**
SEC-02	**Protected Deletion:** An owner attempts to delete a public path that has been used by at least one other rider.	The database rejects the operation. The frontend displays a warning: "Cannot delete: Path is actively used by X riders".	**Pass**
SEC-03	**Privatization Attempt:** An owner switches a path from public to private while it has community activities.	The operation is blocked to preserve visibility and consistency of linked activities.	**Pass**

Table 5.2: Community Integrity Test Cases

5.1.3 Privacy and Workflow Consistency

This subsection validates the advanced workflow logic designed to prevent data duplication and respect user ownership priority during path matching and publication.

ID	Test Scenario	Expected Outcome	Result
WRK-01	**Ownership Priority:** User A has a private path P_{priv}. An identical public path P_{pub} also exists. User A records an activity on this route.	The system detects both paths but prioritizes ownership. The UI automatically links the activity to P_{priv} ("Welcome Back") without prompting to link to the public version.	**Pass**
WRK-02	**Tracking Opt-Out:** User records an activity on a public path but chooses not to appear on the leaderboard (unchecks "Link Activity").	The system creates a new **Private Copy** of the path for the user. The activity is linked to this new private entity, ensuring isolation from public stats.	**Pass**
WRK-03	**Publication Conflict:** User attempts to toggle a private path to "Public", but an identical public path already exists in the system.	The system blocks the direct status change to prevent duplicates. A dialog offers a choice: Merge into the existing path or cancel the operation.	**Pass**
WRK-04	**Merge Resolution:** User accepts the "Merge" option during a publication conflict.	The private path is deleted. All associated activities are transferred to the existing public path. The user receives a success confirmation.	**Pass**
WRK-05	**Passive Cancellation:** User chooses "Keep Private" during a publication conflict.	The publication process is aborted. The path remains private and no changes are made to the database (zero-side-effect action).	**Pass**

Table 5.3: Privacy and Workflow Logic Test Cases

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 6

Installation Instructions

Chapter 7

Effort Spent

Task	Date	Alessio P.	Alberto S.	Luca S.
App Implementation	10/01/2026	3h	2h	2h
	12/01/2026	2h	3h	2h
	15/01/2026	2h	2h	3h
	18/01/2026	1.5h	2h	2h
	Total	**8.5h**	**9h**	**9h**
Document Preparation	22/01/2026	2h	1.5h	1h
	24/01/2026	2h	2h	1.5h
	26/01/2026	1.5h	1.5h	2h
	Total	**5.5h**	**5h**	**4.5h**
TOTAL		**14h**	**14h**	**13.5h**

Table 7.1: Hours of work spent by each author on app implementation and document preparation